

Quản lý nhánh

Tổng quan trong một cái nhìn

Hướng dẫn onboarding tuần đầu
giúp không chỉ developer mà cả designer · planner
hiểu luồng Git/PR qua hình ảnh

Đối tượng Nhân viên mới lần đầu làm quen với Git/PR

Mục tiêu Hiểu luồng "xuất phát từ nhánh nào, hợp nhất vào đâu"

Thời gian đọc Khoảng 15 phút (tóm tắt 5 phút → 1 phút)

Tags

Branch

PR

Cascade

Promotion

Tài liệu này dành cho ai? Nhân viên mới lần đầu làm quen với Git/PR – không chỉ developer mà cả designer và planner

Sau khi đọc xong? Bạn sẽ hình dung được "thay đổi của mình bắt đầu từ nhánh nào và sẽ được hợp nhất vào đâu"

Phải nhớ gì? Không phải câu lệnh, mà là **luồng làm việc**

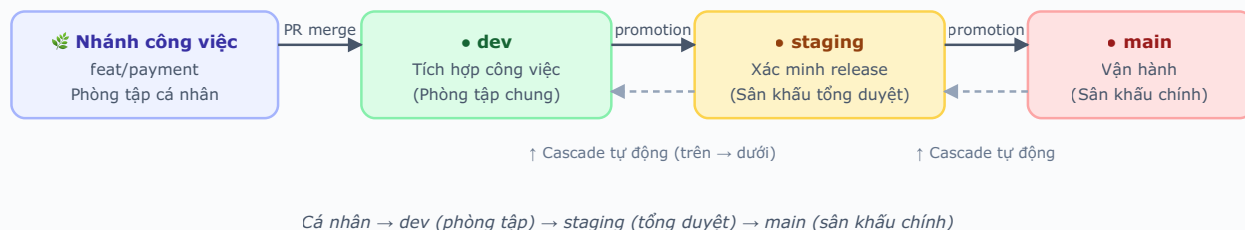
Tóm tắt 5 phút – Phép ẩn dụ cốt lõi

Hãy hình dung các nhánh nơi code được tập hợp như **các sân khấu của một buổi biểu diễn**.

Nhánh	Ẩn dụ	Mô tả
• main	Sân khấu chính thức	Dịch vụ thật mà khách hàng đang thấy. Không được phép xảy ra sự cố
• staging	Sân khấu tổng duyệt	Kiểm tra lần cuối trước khi lên sân khấu chính. "Trạng thái này có thể đưa lên sân khấu được không?"
• dev	Phòng tập	Nơi đầu tiên tất cả công việc mới được tập hợp. Tự do hợp nhất, phá vỡ, làm lại
🌿 feat/fix/...	Phòng tập cá nhân	Không gian riêng để mỗi người tạo ra tác phẩm. Bên ngoài không nhìn thấy

Luồng làm việc: Tác phẩm tạo ra ở phòng tập cá nhân → tập hợp ở **phòng tập chung (dev)** → kiểm tra ở **tổng duyệt (staging)** → đưa lên **sân khấu chính (main)**.

Toàn bộ luồng – Bức tranh lớn



Cách đọc

- Mũi tên nét liền (→) là luồng do con người khởi động: tạo PR, sau khi xác minh thì "thăng cấp" lên bước tiếp theo.
- Mũi tên nét đứt (↑) là luồng do công cụ tự động xử lý: khi nhánh phía trên có thay đổi, các nhánh phía dưới được đồng bộ ngay lập tức.

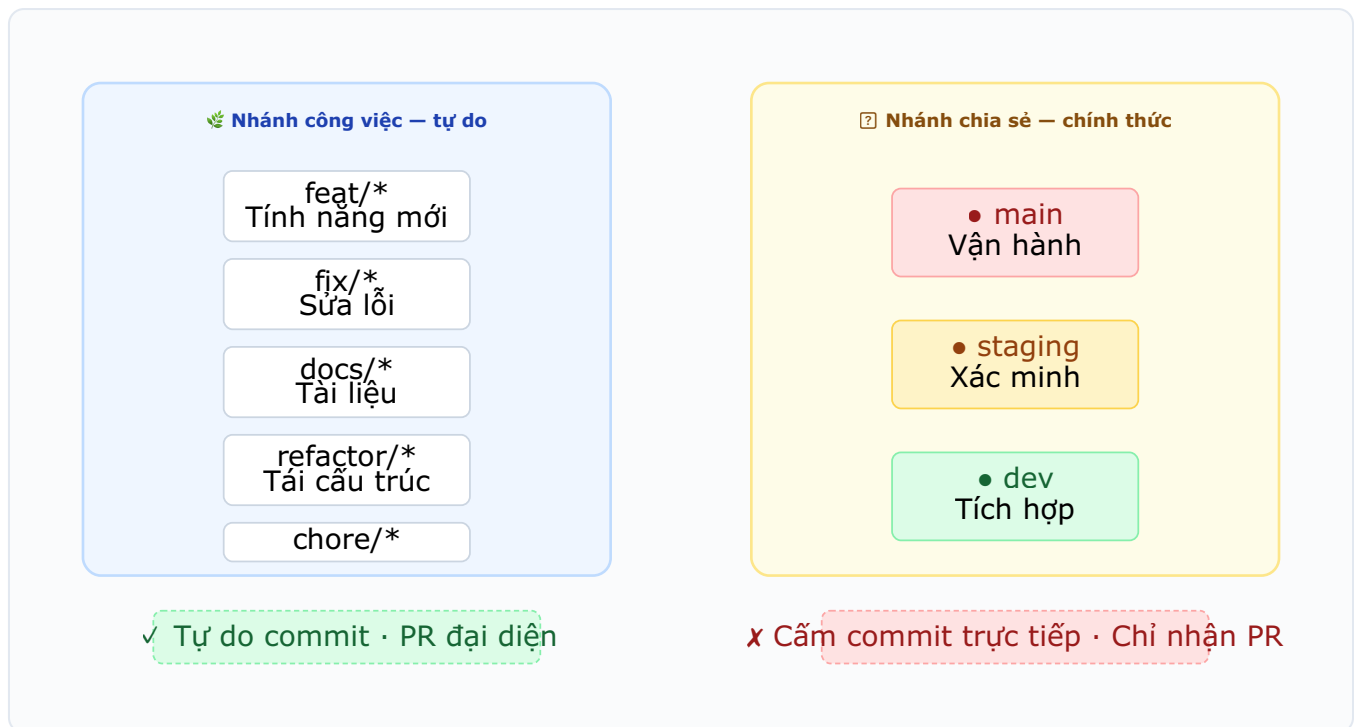
Nhánh	Ai can thiệp?	Thay đổi như thế nào?
Nhánh công việc cá nhân	Bản thân bạn	Lưu (commit) tự do
• dev	Không ai trực tiếp can thiệp	Chỉ qua merge PR (yêu cầu hợp nhất) từ nhánh công việc
• staging	Người phụ trách release	Chỉ qua dev → staging promotion (thăng cấp)
• main	Người phụ trách release	Chỉ qua staging → main promotion

💡 Cốt lõi

dev / staging / main là **nhánh chính thức**. Tuyệt đối không lưu trực tiếp, phải qua PR hoặc promotion mới được đưa vào.

Nhánh chia sẻ vs Nhánh công việc

Các nhánh được chia thành hai loại tính chất lớn.



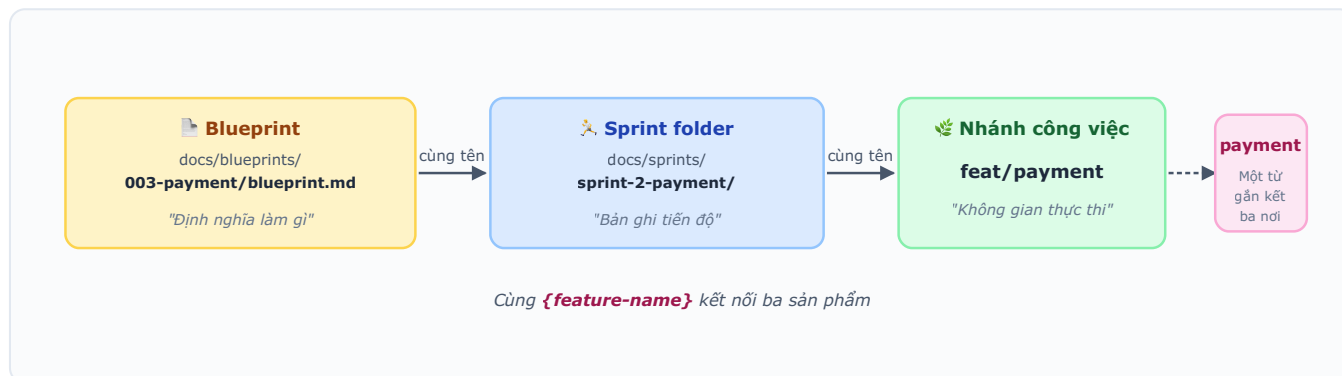
Phân loại	Nhánh nào	Commit trực tiếp?	Khi nào được tạo?
Nhánh chia sẻ	<code>main</code> , <code>staging</code> , <code>dev</code>	✗ Chỉ thay đổi qua PR	Một lần khi khởi tạo repository
Nhánh công việc	<code>feat/*</code> , <code>fix/*</code> , <code>docs/*</code> , <code>refactor/*</code> , <code>chore/*</code>	✓ Tự do commit	Mỗi khi bắt đầu công việc mới

Tại sao phải chia như vậy?

- Nếu can thiệp trực tiếp vào nhánh chia sẻ, **trạng thái mà đồng nghiệp khác đang xem sẽ đột ngột thay đổi**. Thay đổi không qua review là nguồn gốc của sự cố.
- Nhánh công việc là không gian riêng của bạn, có làm hỏng cũng có thể bắt đầu lại mà không áp lực.

Tên nhánh được đặt như thế nào

ASTRA **gắn kết tên của ba sản phẩm thành một từ duy nhất**. Để khi xem PR có thể truy vết ngay "công việc này thuộc sprint nào, bắt nguồn từ blueprint nào".



4 quy tắc đặt tên

Vị trí	Quy tắc	Ví dụ
{feature-name}	Chỉ chữ thường tiếng Anh + dấu gạch nối (kebab-case). Không dùng tiếng Hàn/Việt, chữ hoa, dấu gạch dưới ❌	payment, user-auth, checkout-flow
{NNN}	Số thứ tự blueprint. 3 chữ số (001, 002, ...)	003-payment
{N}	Số thứ tự sprint. Không thêm số 0 (1, 2, 3, ...)	sprint-2-payment
{prefix}	Tính chất của thay đổi (bảng bên dưới)	feat/payment

5 loại prefix

Prefix	Khi nào dùng	Ví dụ
<code>feat/</code>	Thêm tính năng mới (mặc định cho công việc dựa trên blueprint)	<code>feat/payment</code>
<code>fix/</code>	Sửa lỗi	<code>fix/login-error</code> , <code>fix/checkout-crash</code>
<code>docs/</code>	Chỉ thay đổi tài liệu (không sửa code)	<code>docs/onboarding-guide</code>
<code>refactor/</code>	Tái cấu trúc code (hành vi giữ nguyên)	<code>refactor/payment-module</code>
<code>chore/</code>	Build · cấu hình · công cụ (không liên quan đến hành vi dịch vụ)	<code>chore/eslint-bump</code>

Ví dụ truy vết một dòng

 `docs/blueprints/003-payment/blueprint.md` (kế hoạch)
→  `docs/sprints/sprint-2-payment/` (bản ghi tiến độ sprint)
→  `feat/payment` (nhánh công việc)
→  Tiêu đề PR: `feat: thêm tính năng thanh toán`

💡 Tự động quyết định

Tên nhánh công việc được công cụ chuẩn phân tích tính chất thay đổi và tạo tự động. Không cần thuộc lòng — chỉ cần nhớ nguyên tắc **dùng cùng** `{feature-name}` **với blueprint**.

Nhiều việc đồng thời trong một repository

– Cách ly không gian làm việc

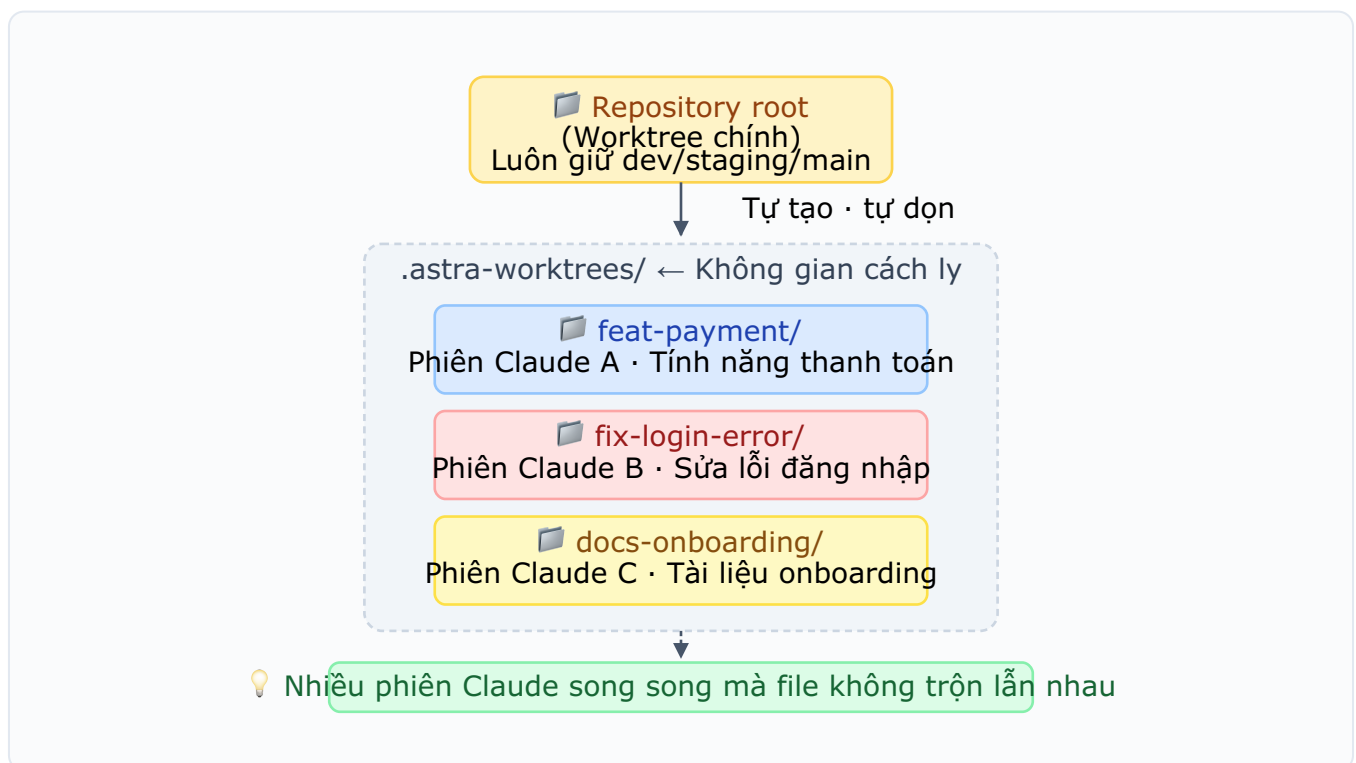
Tại sao cần cách ly?

Có lúc bạn muốn **mở nhiều phiên Claude Code đồng thời** trên cùng repository ở cùng một máy tính để tiến hành các công việc khác nhau (ví dụ: song song tính năng thanh toán + lỗi đăng nhập + tài liệu onboarding).

Với cách thông thường (`git checkout`) thì điều này không thể được.

- Khoảnh khắc phiên A chuyển sang `feat/payment` → tất cả file mà phiên B đang xem sẽ bị thay đổi sang nội dung khác.
- Phiên B sẽ giệt mình tưởng công việc của mình đã biến mất.

Giải pháp: thư mục độc lập cho mỗi nhánh



Mỗi nhánh công việc được tiến hành trong **thư mục riêng biệt trong repository** (git worktree). Vì thư mục khác nhau nên các file cũng độc lập với nhau.

4 hành vi cốt lõi

1. **Không gian làm việc chính chỉ dành cho nhánh chia sẻ** – Thư mục gốc của repository luôn duy trì một trong `dev` / `staging` / `main`. Promotion và đồng bộ hóa diễn ra ở đây.
2. **Quy tắc tên thư mục** – Đổi `/` trong tên nhánh thành `-`: `feat/payment` → `.astra-worktrees/feat-payment/`
3. **Tự động tạo · tự động dọn dẹp** – Khi tạo nhánh công việc, thư mục cũng được tạo cùng. Sau khi merge PR xong sẽ tự động xóa.
4. **Bị gián đoạn thì vẫn giữ nguyên** – Nếu dừng do xung đột · chờ review, thư mục không bị mất. Sau này có thể tiếp tục ngay tại vị trí đó.

💡 Kết quả

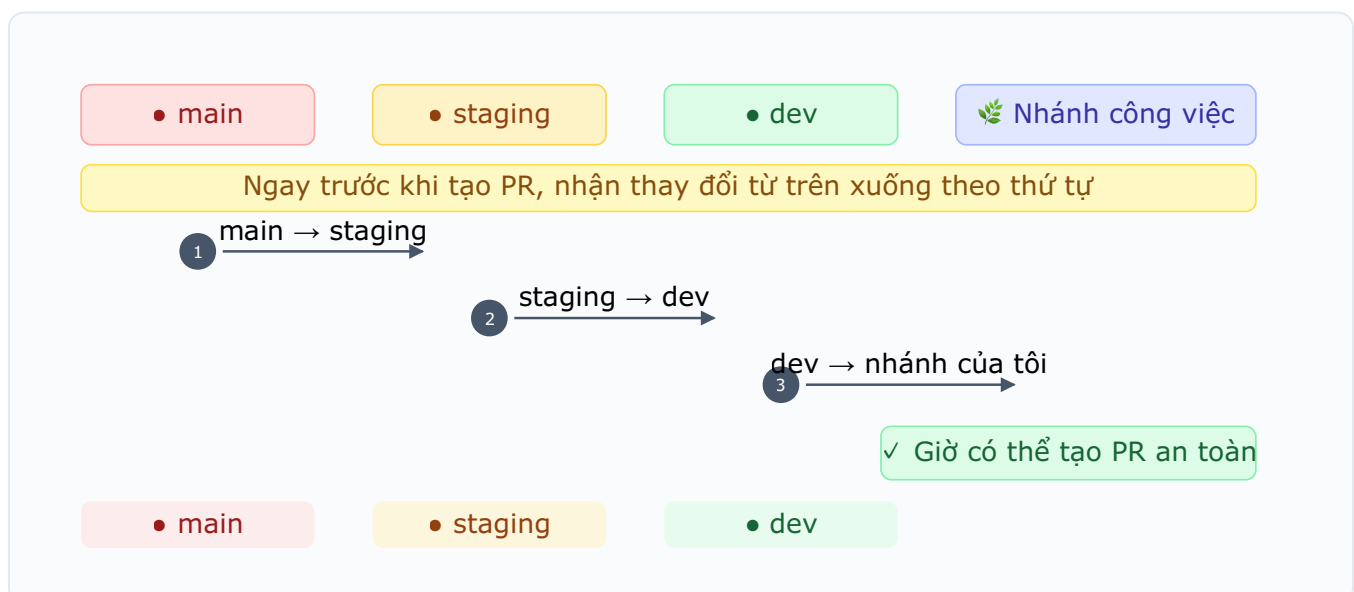
Một người mở hai ba phiên Claude đồng thời để tiến hành công việc thanh toán · đăng nhập · tài liệu cùng lúc mà không xung đột.

0 5

Đồng bộ hóa cascade – Cơ chế an toàn mỗi lần

Điều gì xảy ra?

Ngay trước khi tạo PR, công cụ sẽ chảy thay đổi từ trên xuống dưới theo thứ tự sau.



Tại sao phải làm điều này mỗi lần?

Nếu nhánh `feat/Login` của tôi đã được tách ra từ `dev` cách đây một tuần, thì trong khoảng thời gian đó, `dev` đã nhận rất nhiều code từ các đồng nghiệp khác. Nếu mở PR trong trạng thái đó:

1. **Bùng nổ xung đột** – Luồng review bị gián đoạn và lãng phí thời gian
2. **CI pass** → **vỡ sau khi merge** – Vì được xác minh dựa trên dev cũ nên không thể tin cậy
3. **Hotfix vận hành bị quay ngược** – Nếu sửa lỗi đã vào main không được phản ánh vào dev, lỗi cũ sẽ tái phát ở release tiếp theo

Vai trò của cascade

Ép buộc quy tắc **"tất cả thay đổi của nhánh trên luôn có ở nhánh dưới"** mỗi PR. Công cụ thực hiện mỗi lần để con người không bị quên.

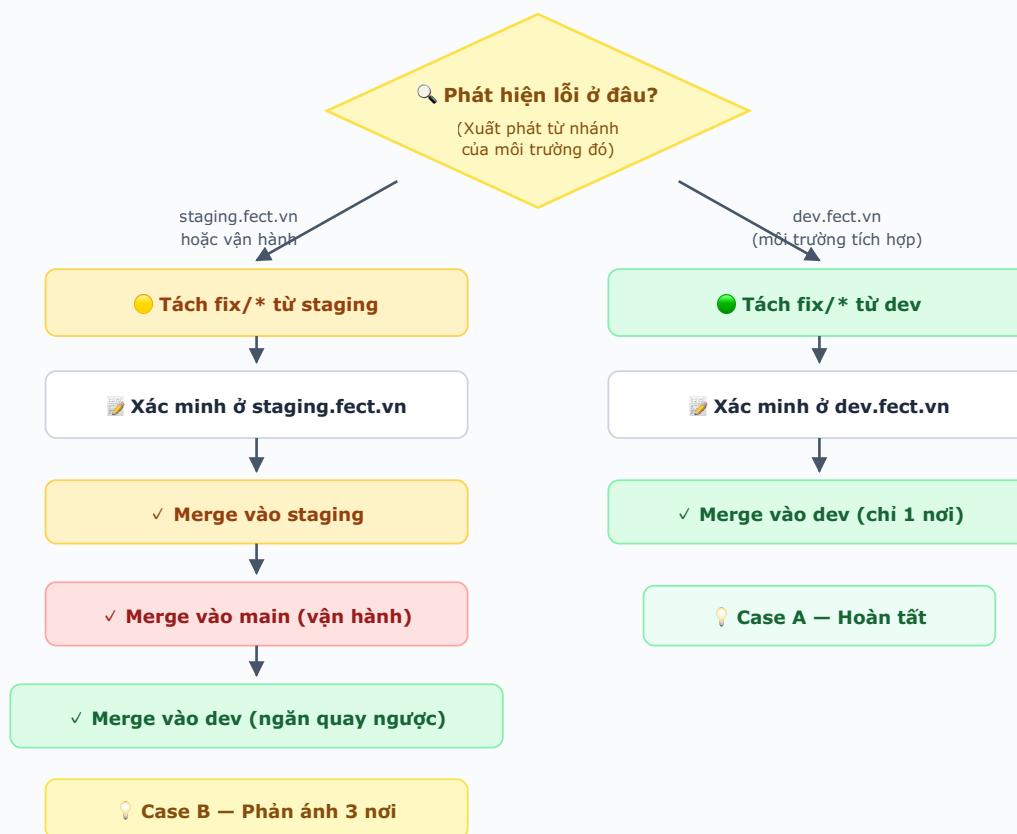
Nếu có xung đột?

Không tự động giải quyết – **người làm phải tự giải quyết trực tiếp**. Vì chỉ có con người mới hiểu được ý định của code.

06

Lý do bugfix khó – Điểm xuất phát khác thì đường đi cũng khác

Cùng một nhánh `fix/*`, nhưng đường merge sẽ hoàn toàn khác nhau tùy theo **nơi phát hiện ra lỗi**.



Quy tắc: nơi thấy lỗi quyết định nơi xuất phát và merge target

📌 Quy tắc một dòng

Xuất phát từ nhánh của môi trường thấy lỗi, và xác minh ở cùng môi trường đó. Chỉ như vậy mới đảm bảo sửa chính xác "trạng thái mà tôi đã thấy".

Case A – Lỗi môi trường dev (lỗi thông thường)

Bước	Nội dung
Phát hiện ở đâu?	dev.fect.vn (môi trường tích hợp phát triển)
Xuất phát từ đâu?	Nhánh dev
Xác minh ở đâu?	dev.fect.vn
Merge vào đâu?	dev (chỉ một nơi)

Luồng giống như phát triển tính năng thông thường.

Case B – Lỗi staging/vận hành (ngay trước · sau release)

Bước	Nội dung
Phát hiện ở đâu?	<code>staging.fect.vn</code> hoặc vận hành (<code>main</code>)
Xuất phát từ đâu?	Nhánh <code>staging</code>
Xác minh ở đâu?	<code>staging.fect.vn</code>
Merge vào đâu?	<code>staging</code> + <code>main</code> + <code>dev</code> cả ba nơi

Tại sao Case B phải merge vào cả ba nơi?

`staging` sắp trở thành `main`. Vì vậy nếu chỉ sửa ở `staging`:

Nơi bị bỏ sót	Hậu quả
Không vào <code>main</code>	Vận hành vẫn ở trạng thái lỗi (khách hàng tiếp tục bị ảnh hưởng)
Không vào <code>dev</code>	Sprint tiếp theo code mới sẽ chảy qua và lỗi cũ tái phát (quay ngược)

⚠ Phải nhớ

Case B = "Nơi sửa + trên (`main`) + dưới (`dev`)" **phản ánh đồng thời ba hướng**. Nếu bỏ sót, vài ngày sau lỗi cũ lại xuất hiện.

Tuyệt đối không được làm

- ❌ Thấy ở `staging` mà xuất phát từ `dev` – Có thể không tái hiện được do khác biệt môi trường
- ❌ Case B chỉ merge vào `staging` – Xảy ra "quay ngược" như trên
- ❌ Đối xử với Case B như Case A, chỉ đi qua `dev` – Đường `staging`→`main` bị phơi nhiễm trong thời gian đó

Promotion (Thăng cấp) – Ai, khi nào, kiểm tra gì

⚠ Quy định riêng của công ty

Bảng dưới **đội nhóm tự điền vào**. Vì là quyết định đưa lên sân khấu chính thức nên người chịu trách nhiệm và thời điểm phải rõ ràng.

Chuyển tiếp	Ai thực hiện?	Khi nào?	Kiểm tra trước
dev → staging	TODO (vd: Sprint Lead)	TODO (vd: Thứ Năm 17h kết thúc sprint)	TODO (vd: E2E pass ở dev, QA sign-off)
staging → main	TODO (vd: Release Manager)	TODO (vd: 10h Thứ Ba hai tuần một lần)	TODO (vd: staging 48 giờ không sự cố, viết release note)

Tăng số phiên bản (SemVer)

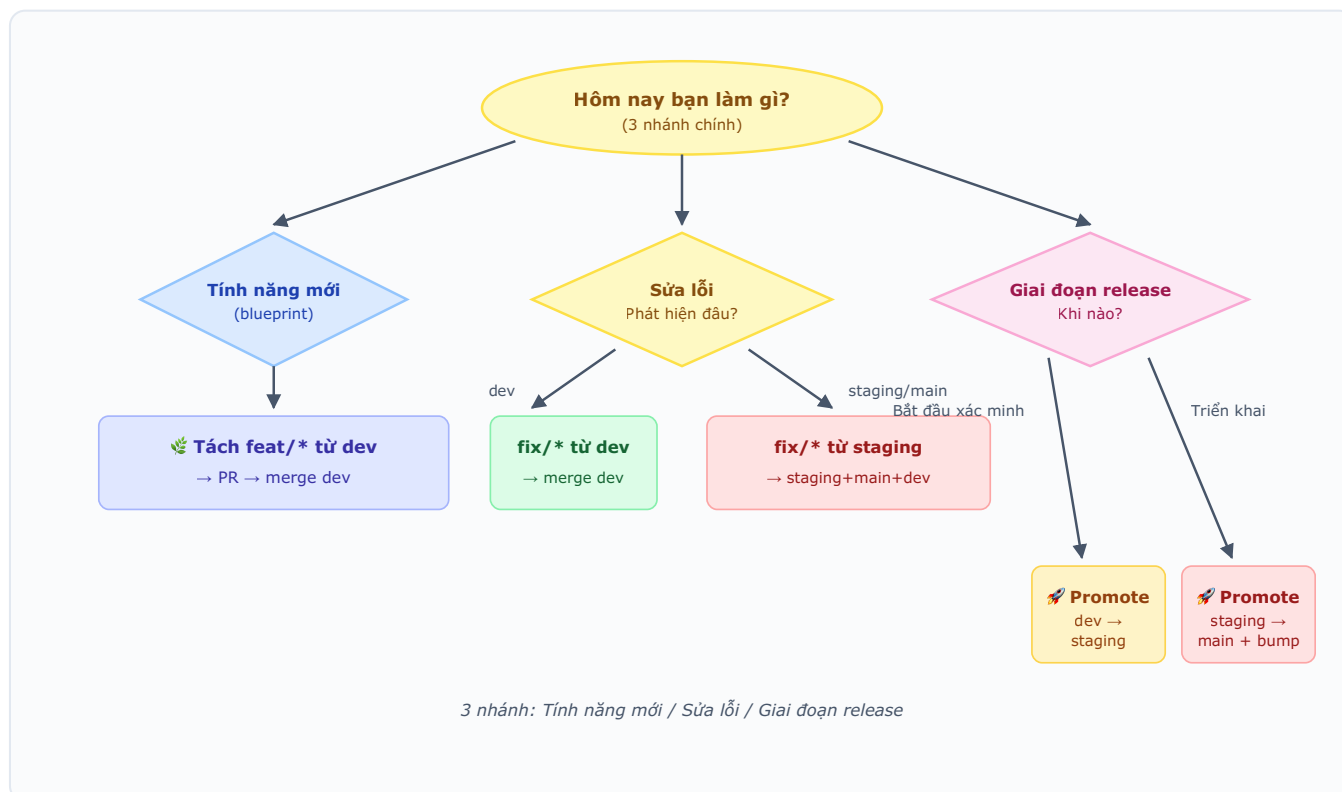
Khi promotion `staging → main`, phiên bản release tăng một bậc. Giống như tạp chí số tăng từ 1 lên 2.

Loại	Thay đổi	Khi nào
patch	1.2.3 → 1.2.4	Khi chỉ có sửa lỗi
minor	1.2.3 → 1.3.0	Khi thêm tính năng mới
major	1.2.3 → 2.0.0	Khi phá vỡ tính tương thích (hiếm)

📌 Tiêu chí phán đoán

Nếu code của người dùng hiện tại bị vỡ thì **major**, nếu là tính năng mới thì **minor**, ngoài ra thì **patch**.

Cây quyết định một trang



Trông phức tạp nhưng rất cuộc chỉ có ba nhánh: **Tính năng mới / Sửa lỗi / Giai đoạn release**.

09

Câu hỏi thường gặp (FAQ)

Q. Đang làm việc thì có việc khác chen ngang, phải làm sao?

Cứ để nhánh công việc hiện tại nguyên trạng, tạo một nhánh công việc mới riêng để tiến hành. Vì không gian làm việc được cách ly nên không ảnh hưởng (tham khảo §4).

Q. Nếu review PR phát hiện Critical issue thì sao?

Nếu còn dù chỉ 1 Critical issue, merge bị chặn. Sau khi sửa hãy cập nhật cùng PR đó.

Q. Có được dùng các lệnh ép buộc như `git push --force` không?

Với nhánh chia sẻ (`main` / `staging` / `dev`) thì **tuyệt đối cấm**. Với nhánh công việc thì có thể trong phạm vi PR của bản thân, nhưng thường không cần thiết.

Q. Lúng túng không biết xuất phát từ đâu?

Hãy ghi nhớ quy tắc **xuất phát từ nhánh của môi trường đã thấy lỗi**. Thấy ở `dev.fect.vn` thì `dev`, thấy ở `staging.fect.vn` hoặc vận hành thì `staging`.

Q. Xung đột cascade xảy ra quá thường xuyên.

Hãy giữ vòng đời nhánh công việc ngắn. Nhánh không merge trên 1 tuần là nguy hiểm. **Hợp nhất nhỏ và thường xuyên** là câu trả lời.

Q. Designer/Planner cũng phải tạo nhánh sao?

Nếu sản phẩm bạn thay đổi (design token, tài liệu kế hoạch) được đưa vào repository thì **đúng vậy**. Tuy nhiên công cụ sẽ đại diện tạo nhánh nên không cần thuộc lòng câu lệnh. Chỉ cần biết luồng là đủ.

10

Danh sách kiểm tra tuần đầu nhập công ty

- ☐ **Ngày 1** – Đọc kỹ tài liệu này, in 1 bản để cạnh bàn
- ☐ **Ngày 2~3** – PR đầu tiên làm cùng mentor. **Tập trung: xuất phát từ đâu và merge vào đâu**
- ☐ **Ngày 4~5** – Quan sát quá trình promotion `dev` → `staging` từ bên cạnh
- ☐ **Tuần 2** – Khi lỗi `staging` đầu tiên xuất hiện, thực hành §6 Case B cùng mentor
- ☐ **Liên tục** – Mỗi khi bí, quay lại tài liệu này. **Thứ phải nhớ không phải câu lệnh mà là luồng**

Last updated: TODO (điền ngày) · Maintainer: TODO (điền người phụ trách)

ASTRA Methodology · Branch Strategy Training v1.0